

Sunrise Dental Clinic System — Assessment Completion Summary

Module: CIS6003 Advanced Programming (WRIT1) | **Student:** Kirisha | **ID:** JF/BSCSD/19/33 | **Course:** BSc SE (Top-Up) **Repository:** <https://github.com/KirishaTharusan/sunrise-dental-clinic> (public, branches `dev` and `release`, both green on CI) **Prepared:** 24 August 2026 | **Last updated:** 25 August 2026

This document confirms, task by task against the assignment brief, what has been completed for the Sunrise Dental Clinic System coursework, lists the most recent changes applied (staff credential rotation, explicit JWT-based authorization, a full report-format validation, and an official cover sheet, title page and navigable Table of Contents added ahead of the report), and gives the exact file/URL locations for every piece of evidence referenced.

1. Two most recent changes applied

1.1 Staff credential rotated

The previously-seeded STAFF account has been replaced everywhere:

	Old	New
Username	nadeesha	kirisha
Password	Nadeesha@123	Kirisha@123
Full name	Nadeesha Wickramasinghe	Kirisha N
Email	nadeesha@sunrisedentalclinic.lk	kirisha@sunrisedentalclinic.lk

Updated in: `database/schema.sql` (a genuine, freshly-generated BCrypt hash

- never a placeholder string), every backend test file that referenced the old username, `frontend/index.html`'s login-hint, `README.md`, `docs/SETUP.md`, `docs/ASSIGNMENT_REPORT.md`, `testing/TEST_CASES.md`, `testing/postman/SunriseDentalClinic.postman_collection.json`, and `diagrams/sequence-02-register-appointment.mmd` (re-rendered). The admin account (`admin` / `admin123`) is unchanged.

1.2 Explicit JWT-based authorization added

Authorization was already JWT-backed internally, but the token was only ever delivered as an invisible HttpOnly cookie. Login now **also** returns the signed token directly in the response body, and this is now documented and demonstrated end-to-end:

- `LoginResponse` gained a `token` field; `AuthController.login()` returns it alongside the existing `dc_token` cookie.
- `JwtAuthenticationFilter` (unchanged - it already supported this) checks the cookie first, then falls back to an `Authorization: Bearer <token>` header - so either mechanism authenticates identically.
- A new `OpenApiConfig` registers a `bearerAuth` security scheme, so Swagger UI's **Authorize** padlock now works: paste the `token` from a login response and every "Try it out" call carries it automatically.
- `testing/postman/SunriseDentalClinic.postman_collection.json`: the Login requests now run a test script that captures `token` into a collection variable, and the whole collection's inherited auth is set to `Authorization: Bearer {{token}}`, alongside Postman's normal cookie jar.
- A new automated test, `AppointmentFlowIntegrationTest.loginResponseTokenWorksAsBearerAuthorization`, logs in, reads the token out of the JSON body with **no cookie involved at all**, and successfully calls a protected endpoint using only the bearer header - proving the mechanism live, not just asserting it exists.
- `docs/ASSIGNMENT_REPORT.md` §3.6 was rewritten to explain the full JWT authorization flow (signing, claims, expiry, validation, both delivery mechanisms) as the report's central security narrative; `docs/SETUP.md` gained a worked curl example and a Swagger "Authorize" walkthrough; four Swagger screenshots were recaptured showing the padlock icons, the `token` field in the schema, and a live executed login response containing a real signed JWT.

The automated suite grew from 25 to **26 tests, all passing** (see §3 below); the corresponding test-count references were updated consistently across `TEST_PLAN.md`, `TEST_CASES.md`, `SETUP.md`, `ASSIGNMENT_REPORT.md`, `testing/evidence/`, and the tests-passing screenshot was recaptured.

1.3 Report format validated against the brief, and the cover sheet added

Every element of `docs/ASSIGNMENT_REPORT.md`'s PDF/DOCX export tooling (`generate-pdf.js`, `generate-docx.js`) was checked line-by-line against the brief's exact formatting clause and corrected where it deviated:

Brief requirement	Before	After
Paper size A4	✅ Already correct	Unchanged

Brief requirement	Before	After
Margins 1.5in left / 1in right, top, bottom	✓ Already correct	Unchanged
Page numbers, bottom right	✓ Already correct	Unchanged
Line spacing 1.5, whole document	✗ Tables used 1.3	Fixed - tables now 1.5 like everything else
Headings 14pt bold	✓ Already correct	Unchanged
Normal text 12pt	✗ Tables (10-11pt) and inline code (10pt) were smaller	Fixed - tables, the report's one blockquote, and every inline <code>code</code> span are now 12pt
Font face Times New Roman	✗ Inline code used a monospace font (Consolas)	Fixed - the entire document, including every inline code span, is now Times New Roman throughout (verified: zero Consolas references remain in the generated <code>.docx</code> XML)

The only deliberate, documented exception is figure/diagram captions (the small italic text under each screenshot and diagram), kept at a smaller 10pt caption style - a universal, conventional academic formatting practice the brief's clause does not address, and distinct from the "Normal" running body text the clause targets. Every other change removes a genuine deviation.

Verified structurally after regenerating both exports: the `.docx`'s raw XML shows `w:pgSz` = A4, `w:pgMar` = exactly 1440/1440/1440/2160 twips (1in/1in/1in/1.5in), the footer's page-number field is right-aligned, 1,182 text runs at 12pt vs. only 51 at 14pt (headings) and 23 at 10pt (captions only), and zero remaining Consolas references. The PDF was visually re-inspected page by page via a headless-Chrome screenshot.

The official university cover sheet, a title/cover page, and a navigable Table of Contents were also added ahead of the report body. Two earlier approaches were tried and rejected for the cover sheet before landing on the final one. The first transcribed it into Markdown tables, which inherited the report's own Times New Roman/12pt formatting - explicitly not what was wanted. The second rasterised the original `Assignment cover sheet.docx` to pixel-exact PNG images - visually perfect, but flagged as the wrong approach because a cover sheet is a **form**: the institution and the student both need to actually fill fields in it, and a picture can't be filled in. The final approach uses Microsoft Word COM automation (`docs/merge-cover-sheet.ps1`) to copy the original cover sheet's real content - the actual table cells, styles and formatting, not a picture of them - into its own Letter-size section at the very start of the DOCX, so it opens as genuinely fillable Word content identical in appearance to the source. Every field is exactly as blank as the

source template - only the institution-prefilled fields (Unit code CIS6003, Unit title, Year 3, Mode of delivery Full Time, Nature of the Assessment "Course work 100%", Learning Outcomes "1,2,3") are filled in, matching the original exactly. Fields only the student can supply - **Name, Student ID, Study period, Lecturer, Topic of the Case Study, Word count, Due date/Time, and the signed Declaration** - remain blank for the student to complete, since inventing any of this on an official declaration would be a genuine correctness problem, not a convenience.

Immediately after the cover sheet, a title/cover page was added (Sunrise Dental Clinic's own brand palette from `frontend/assets/css/styles.css` - navy/teal/amber - rather than an unrelated colour scheme, and no logo, per the brief given for it): "Sunrise Dental Clinic System" as the title, student name Kirisha, student ID JF/BSCSD/19/33, course BSc SE (Top-Up), submission date, "International College of Business and Technology (ICBT)

- Jaffna" with "Cardiff Metropolitan University" underneath as the affiliated awarding university, and a short project-relevant description in place of placeholder text. A genuinely navigable Table of Contents follows (Heading 2-4): a native Word TOC field in the `.docx` (Ctrl+click to jump, updates automatically if headings move), and real clickable internal links with correctly resolved page numbers in the `.pdf` - the PDF pipeline renders the document twice, once to discover which page each heading lands on and once with those numbers baked into the TOC, since they don't exist until the document is paginated. `docs/SETUP.md` §11 has the full three-command regeneration process (`generate-docx.js` → `merge-cover-sheet.ps1` → `generate-pdf.js`) and why the DOCX and PDF are now built by two independent pipelines rather than one generating the other (Word's own PDF export hung indefinitely on this document).

1.4 Originality / plagiarism review

Every section of `docs/ASSIGNMENT_REPORT.md` is original writing authored specifically to describe this project's actual implementation - not adapted or copied from any external template, tutorial, or prior coursework. As a good-faith, honest check (not a substitute for the University's own Turnitin run, which this session has no access to), six distinctive sentences were pulled from different sections of the report - spanning the introduction, the architecture rationale, a design-pattern critique, the testing-philosophy discussion, the JWT/cookie security rationale, and the double-booking business rule - and searched verbatim online. **None returned an exact match anywhere.** Two honest caveats worth knowing before submission:

1. This was a spot-check of a handful of sentences, not an exhaustive sentence-by-sentence scan of the whole ~7,000-word report - a real Turnitin run remains the authoritative check, and is worth running yourself once the cover sheet is filled in.

2. The report **does** deliberately, properly quote the assignment brief itself in a few places (e.g. §1.1's "Students are free to make necessary assumptions...", always in quotation marks with the source named) and cites six real external works in the References section (Gamma et al., Fielding, RFC 7519, etc.) - Turnitin will correctly flag these as quoted/cited matches against the brief PDF and those publications, which is normal, expected, and not plagiarism; markers routinely exclude properly quoted and bibliography material from a similarity score, and it is worth confirming your Turnitin submission settings do the same rather than expecting a literal 0% including quotes.

2. Task-by-task confirmation against the brief

Task A — System design with UML diagrams (20 marks) — Complete

Requirement	Evidence
Use Case diagram, correct actors/use cases/associations	diagrams/use-case-diagram.mmd / .png — 2 actors (Staff, Administrator via generalisation), all 6 brief functionalities + admin-only use cases
<<include>> / <<extend>> used accurately, with reasoning	Same diagram — 2 genuine <<include>>, 3 genuine <<extend>> with fire-condition notes (verified present in the .mmd source)
Class diagram, private/public modifiers, correct relationships	diagrams/class-diagram.mmd / .png — all 7 entities + 5 pattern classes; real o-- (aggregation), *-- (composition), '<
~3 Sequence diagrams	diagrams/sequence-01-login , sequence-02-register-appointment , sequence-03-generate-bill (.mmd / .png)
Assumptions documented	diagrams/README.md §"Design decisions & assumptions" (7 documented assumptions with reasoning); report §1.1, §2
Critical evaluation of the design	docs/ASSIGNMENT_REPORT.md §2.5
Bonus, beyond the brief	ER diagram (er-diagram) matching database/schema.sql exactly; program Flowchart (flowchart) tracing the menu-driven control flow

Task B — Interactive system, design patterns & architecture (40 marks) — Complete

Requirement	Evidence
(i) Distributed application with web services	3-tier: static frontend ↔ Spring Boot REST API (/api/** , 10 controllers, 29 endpoints) ↔ MySQL. Report §3.1
(ii) Appropriate design patterns	5 GoF patterns + DAO/Repository, each with problem/implementation/critical evaluation: Singleton (AppointmentNumberGenerator , AppConfigManager), Builder (AppointmentBuilder), Factory Method (BillFactory), Strategy (PricingStrategy family), Observer (AppointmentObserver / AppointmentEventPublisher). Report §3.3
(iii) Proper database	database/schema.sql : 7 tables, FKs, CHECK constraints, 2 triggers, 1 stored procedure, 1 function, 2 views — all genuinely called from Java (JdbcTemplate in ReportServiceImpl), not decorative. Report §3.4
Validation mechanisms	Layered: client-side, Jakarta Bean Validation, business-rule layer, database layer. Report §3.5
Reports for decision-making	Dashboard KPIs, daily revenue (stored procedure), dentist utilisation (view) — ReportController / ReportServiceImpl
Sophisticated UI, separate windows	13-page frontend (frontend/pages/*.html), Bootstrap 5, no CDN. Report §3.7
Complex functionality (alerts)	Observer-driven simulated Email/SMS/Audit notifications on every appointment lifecycle event. Report §3.3.5
Sessions/cookies	HttpOnly dc_token cookie. Report §3.6
Explicitly added this session	JWT bearer-token authorization (§1.2 above), now the report's primary security narrative

Task C — Testing (20 marks) — Complete

Requirement	Evidence
Test rationale + TDD explanation	testing/TEST_PLAN.md §3 — full red→green→refactor narrative for the pricing/Strategy logic
Test data devised/derived	testing/TEST_PLAN.md §5 — boundary values (4 vs. 5 prior visits, Fri/Sat/Sun dates, same-day past time)
Test plan produced and applied	testing/TEST_PLAN.md (full document)
Test classes created	9 JUnit 5 classes under backend/src/test/java/...
Tests carried out, documented	testing/TEST_CASES.md — 40+ traceable rows (positive/negative/boundary/validation/API/DB/integration), each mapped to its exact automated test method or manual Postman request

Requirement	Evidence
Demonstrate code passes all tests (screen-grab)	testing/screenshots/14-tests-passing.png — real <code>./mvnw test</code> output, Tests run: 26, Failures: 0, Errors: 0, BUILD SUCCESS
Test automation	<code>./mvnw test</code> (one command) + <code>.github/workflows/ci.yml</code> (runs on every push)
Evaluate success/failure incl. lessons learned	docs/ASSIGNMENT_REPORT.md §4.4 — three real, honestly-recorded lessons (2 proactively applied from prior work, 1 genuinely hit and fixed during this build)
Traceability	testing/TEST_CASES.md's "Automated In" column; report §4.5

Current automated total: 26 tests, 0 failures, 0 errors (testing/evidence/*.txt).

Task D — Git, GitHub & version control (20 marks) — Complete

Requirement	Evidence
Public Git/GitHub repository	https://github.com/KirishaTharusan/sunrise-dental-clinic (public)
Several versions, updated with new features applied to the initial upload	21 milestone commits on <code>dev</code> (scaffold → entities → patterns → security → services/controllers → schema → tests → docs → diagrams → frontend → screenshots/exports → this session's JWT/credential update)
Version control techniques demonstrated	docs/GIT_WORKFLOW.md — <code>.gitignore</code> , descriptive milestone commits, a dedicated <code>dev</code> (active) branch separate from <code>release</code> (stable), the <code>mvnw</code> executable-bit fix applied proactively before it could break CI
Workflow (CI/CD) demonstrated, deployment of changes	<code>.github/workflows/ci.yml</code> — builds + runs the full JUnit suite on every push to <code>dev</code> / <code>release</code> / <code>feature/**</code> and on PRs; every run so far has gone green on the first attempt , including the run just triggered by this session's changes
Latest version deployed and demonstrated in the documentation	Both <code>dev</code> and <code>release</code> are at the same latest commit, both pushed, both green; docs/ASSIGNMENT_REPORT.md §5 and this document reference the live repository directly

3. Where everything lives

Deliverable	Location
Assignment report (source)	docs/ASSIGNMENT_REPORT.md
Assignment report	docs/ASSIGNMENT_REPORT.pdf, docs/ASSIGNMENT_REPORT.docx — official cover sheet as page 1, then the report from its own page — A4, margins 1.5in/1in, 1.5

Deliverable	Location
(submission-ready exports)	line spacing throughout (including tables), Times New Roman throughout (including inline code), 14pt bold headings, 12pt body, page numbers bottom-right; format validated and corrected per §1.3
Diagrams	diagrams/ (7 diagrams: Use Case, Class, 3× Sequence, ER, Flowchart — Mermaid source + rendered PNGs)
Database schema	database/schema.sql
Backend source	backend/src/main/java/com/sunrise/dentalclinic/
Automated tests	backend/src/test/java/com/sunrise/dentalclinic/ (9 classes, 26 methods)
Test plan / test case matrix / evidence	testing/TEST_PLAN.md , testing/TEST_CASES.md , testing/evidence/
Postman collection	testing/postman/SunriseDentalClinic.postman_collection.json
Frontend	frontend/ (13 pages)
Setup guide	docs/SETUP.md
Git workflow	docs/GIT_WORKFLOW.md
CI pipeline	.github/workflows/ci.yml

4. Running the system right now

What	URL
Frontend	http://localhost/sunrise-dental-clinic-client/
Backend API	http://localhost:8082/api
Swagger UI	http://localhost:8082/swagger-ui/index.html

Account	Username	Password	Role
Administrator	admin	admin123	ADMIN
Staff	kirisha	Kirisha@123	STAFF

5. Overall status

All four tasks in the brief — A (UML design), B (interactive system, design patterns, architecture), C (testing), and D (Git/GitHub) — are complete, built to the Excellent-band criteria with evidence for every row of the marking table (full mapping in docs/ASSIGNMENT_REPORT.md §6). The credential rotation, explicit JWT bearer authorization, and the report-format validation/cover-sheet addition (§1) are all implemented, verified, and documented. **Outstanding before submission:** the report's cover sheet has five fields only the student can supply - Name, Study period, Lecturer, Due date/Time, and the signed Declaration (§1.3) - and, separately, an in-progress request to host the live system on Render is currently blocked on Render's own account requirement for a payment card on file (confirmed directly against Render's API; no card is charged for free-tier resources, but Render requires one before creating *any* resource, even free ones). The PostgreSQL migration, Flyway setup, and render-profile configuration this needs are already written and committed (backend/src/main/resources/application-render.yml , backend/src/main/resources/db/migration/postgres/V1__init.sql) so the deployment can proceed immediately once that one account-side step is done; only the render.yaml Blueprint (or the equivalent direct API calls) remains to be finished.